Activity

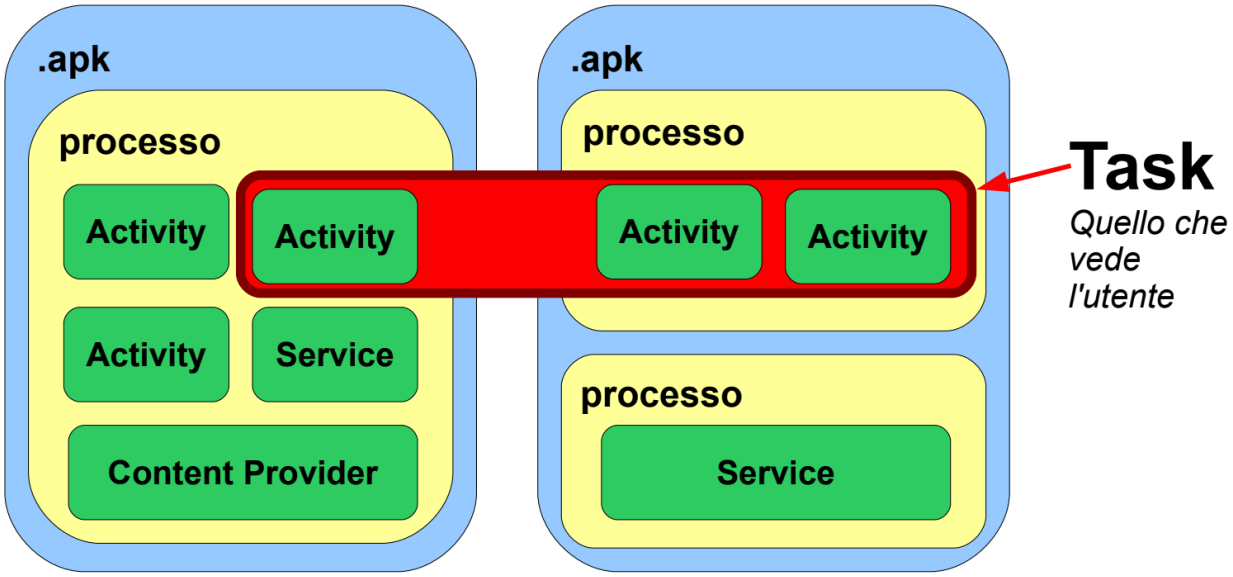
Description	Componenti di un’applicazione / task

Le attività in Android rappresentano le diverse schermate dell'applicazione, ciascuna delle quali offre un'interfaccia utente specifica.

Ogni attività è un componente essenziale dell'applicazione e svolge un ruolo chiave nel ciclo di vita dell'app. Le attività vengono definite mediante classi Java e dichiarate nel file `AndroidManifest.xml`.

Un'attività può essere visualizzata sullo schermo, interagire con l'utente e può essere avviata da altre attività o dal sistema Android stesso.

Quando un'attività viene avviata, il sistema gestisce automaticamente il suo ciclo di vita, passando attraverso diverse fasi come `onCreate`, `onStart`, `onResume`, `onPause`, `onStop` e `onDestroy`.



È comune dichiarare un'attività come attività di avvio nel file `AndroidManifest.xml` specificando l'azione `MAIN` e la categoria `LAUNCHER`. Questa attività sarà quella avviata quando l'applicazione viene avviata dall'utente.

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNC
HER" />
    </intent-filter>
</activity>
```

Nella classe dell'attività, i metodi del ciclo di vita come `onCreate`, `onStart`, `onResume`, ecc., possono essere implementati per eseguire operazioni specifiche in diverse fasi.
Ad esempio:

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    protected void onStart() {
        super.onStart();
        // Operazioni da eseguire quando l'attività sta per d
iventare visibile
    }

    @Override
    protected void onResume() {
        super.onResume();
        // Operazioni da eseguire quando l'attività è in prim
o piano e interagisce con l'utente
    }

    @Override
```

```
    protected void onPause() {
        super.onPause();
        // Operazioni da eseguire quando l'attività sta per p
erdere il focus
    }

    @Override
    protected void onStop() {
        super.onStop();
        // Operazioni da eseguire quando l'attività non è più
visibile
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        // Operazioni da eseguire quando l'attività viene dis
trutta
    }
}
```

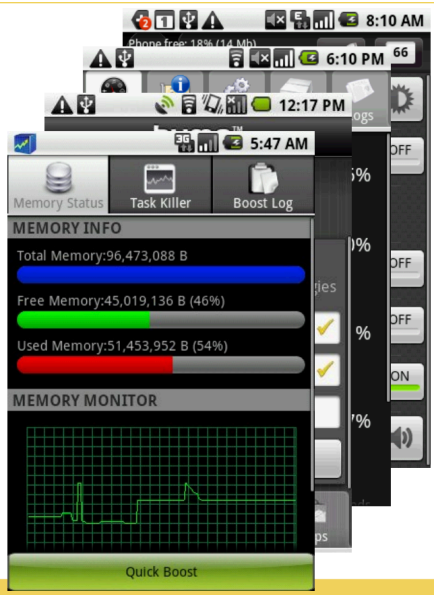
Le attività possono anche essere avviate con `Intent` per comunicare tra di loro. Ad esempio, per avviare un'altra attività:

```
Intent intent = new Intent(this, SecondActivity.class);
startActivity(intent);
```

📌 Le attività possono essere organizzate gerarchicamente. Ad esempio, un'applicazione può avere un'attività principale che avvia altre attività come sottocomponenti.

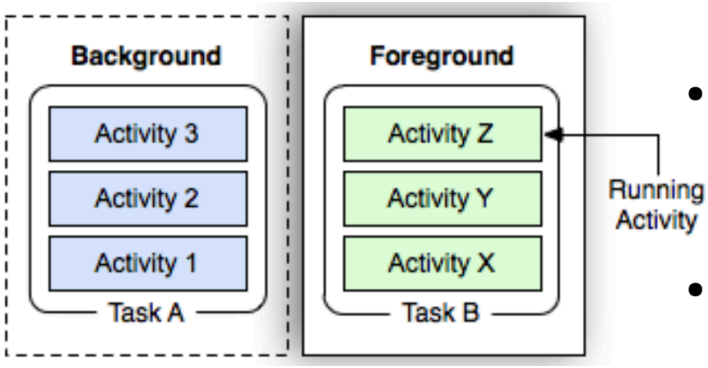
L'utente passa da un'activity all'altra nel corso del suo task

- **Avanti**: usando l'applicazione, si passa di schermata in schermata secondo il flusso del lavoro
- **Indietro**: usando il tasto Back
- **Richiamo**: dalla lista delle “applicazioni recenti”, da notifiche
- **Interruzioni**: da eventi esterni (es.: telefonata in arrivo)
- **Sospensioni**: usando il tasto Home



Si viene quindi a creare uno stack di Activity analogo alla history nei browser.

L'utente vede solo l'activity in cima allo stack ma interagisce solo con la top ma può vedere anche le altre, se la top include delle parti trasparenti



Stack multipli

Un "stack multiplo" si verifica quando ci sono più stack di attività in esecuzione contemporaneamente. Ogni stack di attività rappresenta una pila di attività collegate tra loro, con l'attività in cima alla pila che è attualmente in esecuzione e visibile all'utente.

Scenario di Stack Multiplo:

1. Stack Principale:

- L'applicazione viene avviata, e un'attività principale (ad esempio, `MainActivity`) viene posta nella parte superiore dello stack principale.
- L'utente interagisce con l'applicazione, e nuove attività vengono avviate e aggiunte allo stack principale.

2. Avvio di un'Altra Applicazione:

- L'utente avvia un'altra applicazione. In questo caso, lo stack dell'applicazione originale rimane intatto, mentre l'applicazione secondaria ha il suo stack di attività.

3. App Switching:

- L'utente può passare tra le due applicazioni. Quando l'applicazione originale torna in primo piano, il suo stack di attività diventa attivo. Quando l'applicazione secondaria è in primo piano, il suo stack di attività è attivo.

4. Stack Multiplo:

- In questo modo, abbiamo due stack di attività eseguiti contemporaneamente: uno per l'applicazione originale e uno per l'applicazione secondaria.

Considerazioni Importanti:

- Ogni applicazione ha il proprio stack di attività indipendente.
- L'uso di stack multipli consente un efficiente passaggio tra applicazioni e facilita la gestione del ciclo di vita delle attività.
- Gli stack di attività sono gestiti dal sistema operativo Android in base alle interazioni dell'utente e alle transizioni tra applicazioni.

Gestione dello Stack Multiplo:

- **Intent Flags:**
  - Gli `Intent` possono essere configurati con flag specifici per influenzare il comportamento dello stack di attività. Ad esempio, `FLAG_ACTIVITY_NEW_TASK`

può essere utilizzato per avviare un'attività come nuova attività in uno stack separato.

- **Task Affinity:**
  - È possibile specificare l'affinità di un'attività mediante l'attributo `taskAffinity` nel manifesto. Ciò può influenzare il modo in cui l'attività è associata agli stack di attività.
- **Back Stack:**
  - L'uso corretto della pila delle attività (back stack) tramite la funzione `addToBackStack` durante le transazioni del fragment consente di gestire in modo più efficace il ritorno alle attività precedenti.

launchMode	Effetto
Standard	Crea sempre una nuova istanza dell'activity nel task di destinazione
SingleTop	Se l'activity è top, riavvia l'istanza esistente; altrimenti crea una nuova istanza (che diventa top dello stack)
SingleTask	Può esistere una sola istanza dell'activity. Se ce n'è già una in qualche task, viene riavviata quella. Altrimenti, si crea un nuovo task, che ha l'activity come unico elemento.
SingleInstance	Come sopra, ma se si crea un nuovo task, non consente di creare ulteriori activity al suo interno.

Lancio di un'Activity

```
// Creazione di un Intent con il contesto corrente e la class
e di destinazione
// (SecondActivity.class)
Intent intent = new Intent(context, SecondActivity.class);

// Aggiunta di dati aggiuntivi all'Intent (opzionale)
intent.putExtra("key", "value");
```

```
// Avvio dell'Activity utilizzando startActivity()
startActivity(intent);
```

Nel codice sopra:

- `context` è il contesto corrente, che può essere ottenuto da un'istanza di un'applicazione o da un'altra attività.
- `SecondActivity.class` è la classe Java dell'`Activity` che si desidera avviare.
- `putExtra("key", "value")` consente di aggiungere dati aggiuntivi all'`Intent` in modo che possano essere recuperati dall'`Activity` di destinazione. Questa parte è opzionale e dipende dalle esigenze dell'applicazione.

Dopo aver creato l'`Intent`, puoi avviare l'`Activity` utilizzando il metodo `startActivity(intent)`.

Se desideri ricevere un risultato dalla `SecondActivity`, puoi utilizzare `startActivityForResult()` invece di `startActivity()`. In tal caso, la `SecondActivity` restituirà un risultato, che verrà gestito nel metodo `onActivityResult()` dell'`Activity` chiamante.

Esempio con `startActivityForResult()`:

```
// Creazione di un Intent con il contesto corrente e la class
e di destinazione
// (SecondActivity.class)
Intent intent = new Intent(context, SecondActivity.class);

// Aggiunta di dati aggiuntivi all'Intent (opzionale)
intent.putExtra("key", "value");

// Avvio dell'Activity utilizzando startActivityForResult()
startActivityForResult(intent, requestCode);
```

Interrompere un'Activity

Per interrompere o terminare un'`Activity` in Android, puoi utilizzare il metodo `finish()`. Questo metodo è chiamato per indicare al sistema Android di chiudere l'attività corrente.

Di solito, viene chiamato quando desideri che l'utente torni all'`Activity` precedente o quando desideri chiudere completamente l'applicazione.

Ecco un esempio di come utilizzare il metodo `finish()`:

```
// Chiamare finish() all'interno di un metodo dell'Activity
finish();
```

Puoi chiamare `finish()` da qualsiasi punto all'interno del codice dell'`Activity`. Ad esempio, potresti chiamare `finish()` in risposta a un clic di un pulsante o quando una certa condizione è soddisfatta.

```
// Esempio: Chiamare finish() quando un pulsante viene premut
o
Button closeButton = findViewById(R.id.closeButton);

closeButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        finish();
    }
});
```

Quando `finish()` viene chiamato, l'`Activity` attraversa il ciclo di vita e passa attraverso i metodi `onPause()`, `onStop()`, e infine `onDestroy()`. Se l'`Activity` è stata avviata con `startActivityForResult()`, è possibile passare un risultato utilizzando `setResult()` prima di chiamare `finish()`.

```
// Esempio: Passare un risultato prima di chiamare finish() i
n una ActivityForResult
Intent resultIntent = new Intent();
resultIntent.putExtra("resultKey", "resultValue");
```

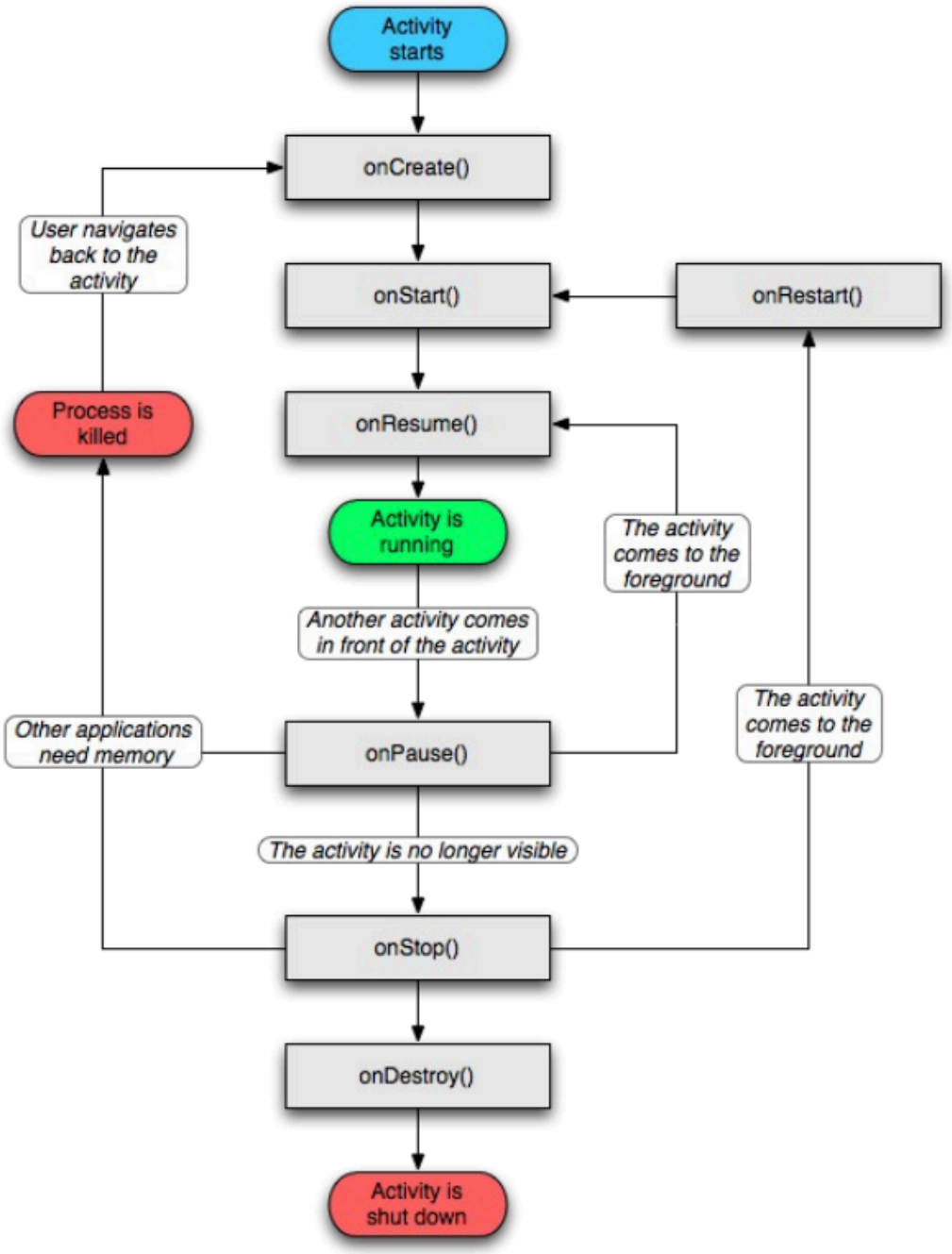
```
setResult(RESULT_OK, resultIntent);
finish();
```

Inoltre, è importante notare che chiamare `finish()` da un'`Activity` la chiuderà, ma non influirà sulle altre attività nello stack.

Se desideri chiudere completamente l'applicazione, dovrai chiudere tutte le attività nello stack. Puoi farlo chiamando `finishAffinity()`:

```
// Chiudere tutte le attività nello stack
finishAffinity();
```

📌 Questo chiuderà tutte le attività associate all'attuale processo dell'applicazione.



- **onCreate()** - inizializziamo lo stato
- **onStart()** - stiamo per essere resi visibili, UI pronta!
- **onResume()** - stiamo per diventare top dello stack, da ora in poi riceviamo input dall'utente

- **onPause()** - un'altra activity sta per diventare top. Salviamo lo stato dell'UI
- **onStop()** - non siamo più visibili, non dobbiamo preoccuparci di tenere la UI aggiornata
- **onDestroy()** - o abbiamo finito (con finish()) o stanno per distruggerci.



La vostra re-implementazione di questi metodi deve chiamare il metodo corrispondente della superclasse

```
@Override
protected void onStart()
{
    super.onStart();
    // codice
}
```

